# Laporan Tugas Information Retrieval

**Mata Kuliah: Information Retrieval**

**Dipersiapkan oleh:**

Tobias Abigail Budiman / 6182201029
Frederick Maximillian Tanupribadi/ 6182201059

**Program Studi Informatika**
**Fakultas Sains**
**Universitas Katolik Parahyangan**

# DAFTAR ISI

# BAB I
# Pendahuluan

## 1.1 Latar Belakang

Information Retrieval merupakan proses pencarian dokumen atau informasi yang relevan dari suatu koleksi dokumen berdasarkan *query* yang diberikan oleh pengguna. Sistem *Information Retrieval* banyak digunakan pada *search engine*, *digital library*, sistem pencarian artikel, dan sistem pencarian dokumen.

Dalam sistem pencarian dokumen dengan metode IR, pencarian tidak dilakukan dengan membaca seluruh isi dokumen satu per satu setiap kali pengguna memasukkan query. Metode dilakukan dengan tahapan preprocessing dan indexing agar proses pencarian lebih efisien. Salah satu struktur indexing yang umum digunakan adalah inverted index, yaitu struktur yang memetakan setiap term ke daftar dokumen tempat term tersebut muncul.

Tugas ini membuat sebuah *search engine* sederhana menggunakan bahasa pemrograman Java. Sistem ini membaca dataset dokumen teks, melakukan *preprocessing*, membangun *inverted index*, serta menyediakan fitur pencarian dokumen. Selain pencarian *single-term*, sistem juga mendukung *Boolean Retrieval* dengan operator `AND`, `OR`, dan `NOT`, serta *Tolerant Retrieval* berupa *wildcard search* dan *typo correction* menggunakan *Levenshtein Distance*.

## 1.2 Rumusan Masalah

Rumusan masalah pada tugas ini adalah sebagai berikut:

1. Bagaimana membaca dan menyimpan kumpulan dokumen teks ke dalam program Java?
2. Bagaimana melakukan preprocessing terhadap dokumen?
3. Bagaimana membangun *inverted index* dari kumpulan dokumen?
4. Bagaimana melakukan pencarian dokumen berdasarkan *query single-term*?
5. Bagaimana mengimplementasikan *Boolean Retrieval* menggunakan operator `AND`, `OR`, dan `NOT`?
6. Bagaimana mengimplementasikan *tolerant retrieval* menggunakan *wildcard search* dan *edit distance*?
7. Bagaimana mengintegrasikan seluruh komponen sistem ke dalam satu *search engine* sederhana?

## 1.3 Tujuan

Tujuan dari tugas ini adalah:

1. Membuat sistem Information Retrieval sederhana menggunakan bahasa pemrograman Java.
2. Membaca dataset dokumen dengan format **doc_id|title|content**.
3. Melakukan preprocessing berupa normalisasi teks, tokenisasi, *stopword removal*, dan *stemming* sederhana.
4. Membangun *inverted index* untuk mempercepat pencarian dokumen.
5. Mengimplementasikan *Boolean Retrieval* menggunakan operator `AND`, `OR`, dan `NOT`.
6. Mengimplementasikan *tolerant retrieval* menggunakan *wildcard search* dan *Levenshtein distance*.
7. Mengintegrasikan seluruh komponen sistem melalui class **SearchEngine**.

## 1.4 Batasan Sistem

Batasan sistem pada implementasi ini adalah:

1. Dataset yang digunakan berupa dokumen teks berbahasa Inggris.
2. Dataset disimpan dalam satu file teks dengan format **doc_id|title|content**.
3. Sistem dibuat menggunakan bahasa Java tanpa menggunakan *library search engine* seperti *Lucene* atau *Elasticsearch*.
4. *Wildcard* search hanya mendukung *trailing wildcard*, misalnya ***comput****.
5. Leading *wildcard* seperti ****tion*** dan *middle wildcard* seperti ***comp*ion*** belum diimplementasikan.
6. *Stemming* yang digunakan masih berupa *simple rule-based stemming*.
7. Ranking dokumen belum diterapkan karena sistem masih berfokus pada *Boolean Retrieval* dan *Tolerant Retrieval*.

# BAB II
# Dasar Teori dan Perancangan

## 2.1 Information Retrieval

Information Retrieval adalah proses menemukan dokumen yang relevan dari suatu koleksi dokumen berdasarkan *query* yang diberikan oleh pengguna. Pengguna memasukkan *query* berupa kata atau pola tertentu, kemudian sistem mengembalikan dokumen yang dianggap sesuai.

Sistem Information Retrieval biasanya terdiri atas beberapa tahapan utama, yaitu:

1. Pengumpulan dokumen.
2. Preprocessing dokumen.
3. Pembuatan index.
4. Pemrosesan query.
5. Pengambilan hasil dokumen.

Dalam tugas ini, sistem difokuskan pada pembuatan search engine sederhana berbasis inverted index.

## 2.2 Preprocessing

Preprocessing adalah tahap awal untuk membersihkan dan menyiapkan teks sebelum dimasukkan ke dalam index. Tujuannya adalah mengubah teks mentah menjadi kumpulan token yang lebih mudah diproses oleh sistem.

Tahapan preprocessing yang digunakan pada sistem ini meliputi:

1. **Lowercasing**
   Mengubah seluruh huruf menjadi huruf kecil agar kata seperti **Computer** dan **computer** dianggap sama.
2. **Punctuation removal**
   Menghapus tanda baca dan simbol agar hanya tersisa huruf, angka, dan spasi.
3. **Tokenization**
   Memecah teks menjadi potongan kata atau token.
4. **Stopword removal**
   Menghapus kata-kata umum yang dianggap paling kurang bermakna dalam pencarian, seperti **the**, **is**, **and**, **of**, dan **to**.
5. **Simple stemming**
   Mengubah kata menjadi bentuk dasar, misalnya **documents** menjadi **document** dan **searching** menjadi **search**.

## 2.3 Inverted Index

Inverted index adalah struktur data yang memetakan term ke daftar dokumen tempat term tersebut muncul. Struktur ini digunakan agar pencarian tidak perlu membaca seluruh dokumen secara langsung

Bentuk Umum inverted index adalah:

Term → posting list

Contoh:
- Computer → [2, 7, 12, 51]
- Retrieval → [1, 66, 71, 72]
- Document → [1, 34, 67, 68]

Pada sistem ini, inverted index direpresentasikan menggunakan:

Map<String, Set<Integer>>

Keterangan:
- String untuk menyimpan term
- Set<Integer> untuk menyimpan daftar doc_id

Penggunaan Set<Integer> bertujuan agar satu dokumen tidak masuk lebih dari satu kali untuk term yang sama.

## 2.4 Tolerant Retrieval

Tolerant Retrieval adalah teknik pencarian yang memungkinkan sistem menangani *query* yang tidak sama persis. Pada sistem ini, tolerant retrieval terdiri dari dua pendekatan:

1. **Wildcard Search**
   Digunakan untuk *query* dengan karakter "*", misalnya:
   `comput*`

   *Query* tersebut dapat dianggap mirip dengan term seperti:
   - computer
   - computing
   - computation
   - computational

2. **Edit Distance / Spelling Correction**
   Digunakan untuk menangani kesalahan ketikan pada *query*, misalnya:
   - `computr` → computer
   - `retrival` → retrieval
   - `documnt` → document

   Metode edit distance yang digunakan adalah Levenshtein Distance, yaitu perhitungan jumlah minimum operasi insert, delete, dan replace untuk mengubah satu kata menjadi kata lain.

## 2.5 Design Pattern

Sistem ini menggunakan beberapa design pattern agar struktur kode lebih rapi.

### 2.5.1 Facade Pattern

Facade Pattern digunakan pada class **SearchEngine**. Class ini menjadi pintu utama untuk menjalankan sistem pencarian. Class **SearchEngine** akan memastikan class **Main** tidak perlu memanggil seluruh komponen internal secara langsung.

### 2.5.2 Strategy Pattern

Strategy Pattern digunakan pada dua bagian:

1. Preprocessing
```
Preprocessor
└── BasicPreprocessor
```
2. Tolerant retrieval:
```
TolerantSearchStrategy
├── WildcardSearchStrategy
└── EditDistanceStrategy
```

Dengan pattern ini, strategi pemrosesan dapat dipisahkan dan dikembangkan secara modular.

### 2.5.3 Composite / Interpreter Pattern

Pattern ini digunakan untuk *Boolean retrieval*. Struktur yang digunakan adalah:

```
QueryExpression
├── TermExpression
├── AndExpression
├── OrExpression
└── NotExpression
```

## 2.6 Pembagian Tugas Anggota Kelompok

Pengerjaan tugas dilakukan oleh dua anggota kelompok dengan pembagian peran berdasarkan modul utama pada sistem *Information Retrieval*. Peran 1 dikerjakan secara bersama karena menjadi fondasi utama sistem, sedangkan Peran 2 dan Peran 3 dibagi sesuai fokus implementasi masing-masing anggota.

| Anggota | Peran | Modul yang Dikerjakan | Keterangan |
|---|---|---|---|
| Tobias dan Frederick | Peran 1- Data, Preprocessing, dan Indexing | **Document**, **DocumentStore**, **TextUtil**, **Preprocessor**, **BasicPreprocessor**, **InvertedIndex**, **dokumen.txt** | Dikerjakan bersama karena modul ini menjadi dasar untuk seluruh fitur pencarian. Modul ini bertanggung jawab membaca dataset, membersihkan teks, membuat token, dan membangun inverted index |

| | | | |
|---|---|---|---|
| Frederick | Peran 2 - Boolean Retrieval | **QueryExpression**, **TermExpression**, **AndExpression**, **OrExpression**, **NotExpression**, **QueryParser**, **BooleanEngine** | Bertanggung jawab membuat fitur pencarian *Boolean* menggunakan operator `AND`, `OR`, dan `NOT`, termasuk parsing query dan evaluasi expression |
| Tobias | Peran 3 - Tolerant Retrieval | **TolerantSearchStrategy**, **TrieNode**, **Trie**, **WildcardSearchStrategy**, **EditDistanceStrategy**, **TolerantRetrieval** | Bertanggung jawab membuat fitur *tolerant*, yaitu *wildcard search* seperti `comput*` dan *typo correction* menggunakan *Levenshtein Distance* |
| Tobias dan Frederick | Integrasi dan Pengujian | **SearchEngine**, **Main**, pengujian *query*, README, laporan, dan video demo | Integrasi dilakukan bersama agar modul *indexing*, *Boolean retrieval*, dan *tolerant retrieval* dapat berjalan dalam satu sistem pencarian |

# BAB III
# Implementasi Sistem

## 3.1 Struktur Project

Struktur project yang digunakan adalah sebagai berikut:

```
IR_Search_Engine/
├── data/
│   └── dokumen.txt
├── src/
│   ├── Main.java
│   ├── core/
│   │   ├── Document.java
│   │   ├── DocumentStore.java
│   │   ├── InvertedIndex.java
│   │   └── SearchEngine.java
│   ├── preprocessing/
│   │   ├── Preprocessor.java
│   │   └── BasicPreprocessor.java
│   ├── booleanquery/
│   │   ├── QueryExpression.java
│   │   ├── TermExpression.java
│   │   ├── AndExpression.java
│   │   ├── OrExpression.java
│   │   ├── NotExpression.java
│   │   ├── QueryParser.java
│   │   └── BooleanEngine.java
│   ├── tolerant/
│   │   ├── TolerantSearchStrategy.java
│   │   ├── TolerantRetrieval.java
│   │   ├── WildcardSearchStrategy.java
│   │   ├── EditDistanceStrategy.java
│   │   ├── Trie.java
│   │   └── TrieNode.java
│   └── util/
│       └── TextUtil.java
└── README.md
```

## 3.2 Dataset

Dataset disimpan pada file:

data/dokumen.txt

Format dokumen yang digunakan adalah:

doc_id|title|content

Contoh:

1|Information Retrieval|Information retrieval is the process of finding relevant documents from a large collection.

Dataset berisi 100 dokumen pendek berbahasa Inggris dengan topik yang berkaitan dengan computer science, information retrieval, artificial intelligence, database, cybersecurity, data mining, dan teknologi lainnya

## 3.3 Implementasi Class Document

Class **Document** digunakan untuk merepresentasikan satu dokumen. Class ini memiliki tiga atribut utama:

1. Id
2. Title
3. Content

Fungsi class ini hanya sebagai penyimpanan data dokumen. Class ini tidak melakukan proses preprocessing, indexing, atau searching.

## 3.4 Implementasi Class DocumentStore

Class **DocumentStore** digunakan untuk membaca dan menyimpan seluruh dokumen dari file teks. Struktur data yang digunakan adalah:

Map<Integer, Document>

Artinya, setiap dokumen disimpan berdasarkan doc_id

Method utama pada class ini adalah:

- loadFromFile(String filePath)
- getDocumentById(int id)
- getAllDocuments()
- getAllDocumentIds()
- size()

Method **loadFromFile()** membaca file baris per baris, kemudian memecah setiap baris berdasarkan separator “|”. Setelah itu, sistem mengambil doc_id, title, dan content, lalu membuat object **Document**

## 3.5 Implementasi Class TextUtil

Class **TextUtil** berisi method bantuan untuk pengolahan teks. Method yang digunakan antara lain:

- normalizeText(String text)
- normalizeToken(String token)
- simpleStem(String token)
- isBooleanOperator(String token)
- containsBooleanOperator(String query)
- containsWildcard(String query)

**normalizeText()** digunakan untuk membersihkan teks panjang atau paragraf. Method ini mengubah teks menjadi lowercase, menghapus tanda baca, dan merapikan spasi.

**normalizeToken()** digunakan untuk membersihkan satu token. Method ini mengubah token menjadi lowercase, menghapus karakter selain huruf dan angka, lalu menjalankan stemming sederhana.

**simpleStem()** digunakan untuk menghapus akhiran tertentu dari kata. Contoh hasil stemming:
- Documents → document
- Systems → system
- Searching → search
- Indexed → index

**containsWildcard()** digunakan untuk mengecek apakah *query* mengandung karakter “*”.
**isBooleanOperator()** digunakan untuk mengecek apakah token merupakan operator *Boolean*.
**containsBooleanOperator()** digunakan untuk mendeteksi apakah *query* mengandung operator *boolean*.
**containsWildcard()** digunakan untuk mengecek apakah *query* mengandung karakter “*”

## 3.6 Implementasi Interface Preprocessor dan BasicPreprocessor
Interface **Preprocessor** digunakan sebagai kontrak untuk proses preprocessing.
Interface ini memiliki method:
List<String> process(String text);

Method ini menerima teks mentah dan mengembalikan daftar token bersih.

Class **BasicPreprocessor** mengimplementasikan interface **Preprocessor**

Tahap yang dilakukan pada method **process()** adalah:
1. Menerima content dokumen.
2. Melakukan normalisasi teks menggunakan **TextUtil.normalizeText()**.
3. Memecah teks menjadi token menggunakan **split(“\\s+”)**.
4. Menormalisasi setiap token menggunakan **TextUtil.normalizeToken()**.
5. Menghapus token kosong.
6. Menghapus stopword.
7. Mengembalikan token bersih dalam bentuk **List<String>**.

Contoh:
- Input:
  Information retrieval is the process of finding relevant documents.
- Output:
  [Information, retrieval, process, finding, relevant, document]

## 3.7 Implementasi Class InvertedIndex
Class **InvertedIndex** digunakan untuk membangun index utama sistem pencarian.

Struktur data yang digunakan adalah:
Map<String, Set<Integer>>

Artinya:
Term → posting list

Method utama pada class ini adalah:
- addDocument(int docId, List<String> tokens)
- getPostingList(String term)
- containsTerm(String term)
- getVocabulary()
- getVocabularySize()
- printSampleIndex(int limit)

Method **addDocument()** akan memasukkan token hasil preprocessing ke dalam *inverted index*. Jika *term* belum ada, sistem akan membuat *posting list* baru. Jika *term* sudah ada, **doc_id** ditambahkan ke *posting list*.

Method **getPostingList()** mengembalikan daftar dokumen untuk term tertentu. Jika term tidak ditemukan, method ini mengembalikan **Set** kosong agar tidak terjadi error.

## 3.8 Implementasi Boolean Retrieval

*Boolean Retrieval* diimplementasikan pada package *booleanquery*.

### 3.8.1 QueryExpression

**QueryExpression** adalah interface utama untuk semua expression *Boolean*.

Method:
Set<Integer> evaluate(InvertedIndex index, Set<Integer> allDocumentIds);

Setiap expression akan mengembalikan daftar **doc_id** hasil evaluasi.

### 3.8.2 TermExpression

**TermExpression** merepresentasikan satu *term* dalam *query*.

Contoh:
Computer

Evaluasi dilakukan dengan mengambil *posting list* dari *inverted index*.

### 3.8.3 AndExpression

**AndExpression** merepresentasikan operasi **AND**.

Contoh:

Information AND retrieval

Cara kerja:
posting(information) AND posting(retrieval)

Implementasinya menggunakan operasi intersection dengan **retainAll()**.

### 3.8.4 OrExpression

**OrExpression** merepresentasikan operasi OR

Contoh:
Computer OR software

Cara Kerja:
posting (computer) ∪ posting(software)

Implementasinya menggunakan operasi union dengan addAll().

### 3.8.5 NotExpression

**NotExpression** merepresentasikan operasi `NOT`.

### 3.8.6 QueryParser

**QueryParser** bertugas mengubah *query* user menjadi expression tree.

Tahapan parser:

1. Melakukan tokenisasi *query*.
2. Memisahkan tanda kurung agar dapat diproses sebagai *token*.
3. Menambahkan operator `AND` implisit sebelum `NOT` untuk *query* seperti:
   computer NOT library
   *Query* tersebut diproses sebagai:
   computer AND NOT library
4. Mengubah *query* dari bentuk infix menjadi postfix
5. Membangun expression tree dari postfix

Parser mendukung operator:
`AND`
`OR`
`NOT`

Parser juga mendukung tanda kurung, misalnya:
information AND ( retrieval OR document )

### 3.8.7 Boolean Engine

**BooleanEngine** menjadi class penghubung untuk *Boolean Retrieval*.

Alur kerja:
query Boolean
↓
QueryParser.parse(query)
↓
QueryExpression
↓
evaluate(index, allDocumentIds)
↓
Set<Integer> result

Contoh *query* yang didukung:
information AND retrieval
computer OR software
computer NOT library
information AND ( retrieval OR document )

## 3.9 Implementasi Trie untuk Wildcard Search

Wildcard search menggunakan struktur data Trie. Trie digunakan untuk menyimpan vocabulary dari inverted index agar sistem dapat mencari semua term dengan prefix tertentu.

### 3.9.1 Class TrieNode

Class TrieNode merepresentasikan satu node dalam Trie. Atribut yang digunakan adalah:
- children
- endOfWord
- term

**children** menyimpan cabang karakter berikutnya menggunakan **Map<Character, TrieNode>**. Penggunaan **HashMap** membuat Trie lebih fleksibel dibanding array 26 huruf.

**endOfWord** menandakan apakah sebuah node akhir, sehingga ketika sistem menemukan node akhir saat prefix search, sistem dapat mengambil kata lengkapnya

### 3.9.2 Class Trie

Class **Trie** memiliki beberapa method utama:
- insert(String term)
- search(String term)
- getTermsWithPrefix(String prefix)
- collectTerms(TrieNode node, List<String> result)

**insert()** digunakan untuk memasukkan term ke dalam Trie.
**search()** digunakan untuk mengecek untuk mencari semua term yang diawali prefix tertentu.

**getTermWithPrefix()** digunakan untuk mencari semua term yang diawali prefix tertentu.
**collectTerms()** digunakan untuk menelusuri semua cabang dari node prefix dan mengumpulkan semua term yang ditemukan.

Contoh:
getTermsWithPrefix("comput")

Hasil:
[computer, computing, computation, computational]

## 3.10 Implementasi Class WildcardSearchStrategy

Class WildcardSearchStrategy mengimplementasikan interface **TolerantSearchStrategy**.

Class ini menangani *query wildcard* dengan bentuk *trailing wildcard*, misalnya:

- `comput*`
- `retriev*`
- `doc*`

Alur kerja wildcard search adalah:

1. *Query* diterima,misalnya `comput*.`
2. Sistem mengecek apakah *query* berakhir dengan “*”.
3. Prefix diambil, yaitu `comput.`
4. Trie mencari semua term yang diawali prefix tersebut.
5. Untuk setiap matched term, sistem mengambil *posting list* dari *inverted index*.
6. Semua *posting list* digabungkan menjadi hasil akhir.

Wildcard search pada implementasi ini belum mendukung *leading wildcard* seperti `*tion` dan *middle wildcard* seperti `comp*ion.`

## 3.11 Implementasi Class EditDistanceStrategy

Class **EditDistanceStrategy** digunakan untuk menangani *typo correction*. Class ini juga mengimplementasikan **TolerantSearchStrategy**.
Contoh kalau *typo query*:

- Computr → computer
- retrival → retrieval
- documnt → document

Sistem akan membandingkan *query* tersebut dengan seluruh vocabulary term menggunakan *Levenshtein Distance***.**

Method utama:

- search(String query, InvertedIndex index)
- suggest (String, InvertedIndex index)

- levenshteinDistance(String a, String b)
- getLastSuggestion()
- clearLastSuggestion()

Alur Kerja:

1. *Query* dinormalisasi.
2. *Query* dibandingkan dengan setiap term pada vocabulary.
3. Sistem menghitung Levenshtien Distance
4. Term dengan distance terkecil dipilih sebagai suggestion.
5. Jika distance masih berada dalam batas **maxDistance**, sistem mengambil posting list dari term suggestion.
6. Hasil dokumen dikembalikan

Class ini mencari term terdekat dari vocabulary berdasarkan Levenshtien Distance. *Query* dinormalisasi terlebih dahulu, kemudian dibandingkan dengan seluruh vocabulary. Jika jarak edit terkecil masih berada dalam batas **maxDistance,** sistem memberikan suggestion dan mengembalikan *posting list* dari *term* tersebut

## 3.12 Implementasi Class TolerantRetrieval

Class **TolerantRetrieval** digunakan sebagai penghubung antara **WildcardSearchStrategy** dan **EditDistanceStrategy**.

Method utama:

- buildWildcardTrie(InvertedIndex index)
- searchWildcard(String query, InvertedIndex index)
- searchWithCorrection(String query, InvertedIndex index)
- suggestCorrection(String query, InvertedIndex index)
- getLastSuggestion()

Class ini membuat **SearchEngine** tidak perlu memanggil *wildcard* dan *edit distance* secara langsung.

## 3.13 Implementasi Class SearchEngine

Class **SearchEngine** berperan sebagai facade utama dari sistem pencarian.

Method utama:

- buildIndex(String filePath)
- search(String query)
- printResults(Set<Integer> resultIds)
- printSampleIndex(int limit)
- getDocumentCount()
- getVocabularySize()
- getLastSuggestion()

Method **buildIndex()** digunakan untuk membaca dokumen, melakukan preprocessing, membangun *inverted index*, dan membangun *trie* untuk *wildcard search*.

Method **search()** digunakan untuk menentukan jenis *query*. Alur pencarian adalah:

1. Jika *query* mengandung *wildcard*, gunakan **TolerantRetrieval.searchWildcard()**.
2. Jika *query* mengandung operator *Boolean*, gunakan **BooleanEngine.search()**.
3. Jika *query* adalah *single-term* dan *term* ada di *index*, ambil *posting list* langsung.
4. Jika *term* tidak ditemukan, gunakan **EditDistanceStrategy** untuk *typo correction*.

## 3.14 Implementasi Class Main

Class **Main** digunakan sebagai antarmuka terminal sederhana. Program menampilkan menu:

1. Search query
2. Show sample inverted index
3. Exit

Menu pertama digunakan untuk memasukkan *query* pencarian. Menu kedua digunakan untuk menampilkan sebagian isi *inverted index*. Menu ketiga digunakan untuk keluar dari program.

## 3.15 Arsitektur Alur Sistem

Arsitektur sistem pada program *Information Retrieval* ini dibagi menjadi beberapa modul utama. Setiap modul memiliki tugas yang berbeda agar struktur program lebih rapi, modular, dan mudah dikembangkan.

### 3.15.1 Modul Utama Sistem

Sistem terdiri dari beberapa modul utama sebagai berikut

| Modul | Class / Package | Fungsi |
|---|---|---|
| Modul Dokumen | **Document**, **DocumentStore** | Membaca dan menyimpan dokumen dari file dataset. |
| Modul Preprocessing | **Preprocessor**, **BasicPreprocessor**, **TextUtil** | Membersihkan teks dan mengubah isi dokumen menjadi token bersih. |
| Modul Indexing | **Inverted Index** | Membentuk *inverted index* berupa mapping antara *term* dan daftar **doc_id**. |
| Modul Boolean Retrieval | **booleanquery** | Memproses *query* dengan operator AND, OR, dan NOT |
| Modul Tolerant Retrieval | **tolerant** | Menangani *wildcard search* dan *typo correction*. |
| Modul Integrasi | **SearchEngine** | Menghubungkan seluruh modul pencarian |

| | | |
|---|---|---|
| Modul Antarmuka | **Main** | Menyediakan menu terminal untuk user. |

### 3.15.2 Alur Pembacaan Dokumen

Alur pembacaan dokumen dilakukan sebagai berikut:

```
data/dokumen.txt
     ↓
DocumentStore
     ↓
Document
```

Penjelasan:

1. Dataset disimpan dalam file **data/dokumen.txt**.
2. Setiap baris dokumen memiliki format:
   doc_id|title|content
3. **DocumentStore** membaca file tersebut baris per baris.
4. Setiap baris diubah menjadi objek **Document**.
5. Objek **Document** menyimpan tiga informasi utama:
   id
   title
   content

### 3.15.3 Alur Preprocessing

Setelah dokumen dibaca, isi dokumen diproses oleh **BasicPreprocessor**.

Alurnya:

```
Document content
     ↓
TextUtil.normalizeText()
     ↓
Tokenization
     ↓
TextUtil.normalizeToken()
     ↓
Stopword removal
     ↓
Simple stemming
     ↓
List<String> tokens
```

Tahapan preprocessing:

1. **Normalisasi teks**
   Mengubah teks menjadi lowercase, menghapus tanda baca, dan merapikan spasi.

2. **Tokenisasi**
   Memecah teks menjadi kata-kata menggunakan spasi.
3. **Normalisasi token**
   Membersihkan setiap token dari karakter yang tidak diperlukan.
4. **Stopword removal**
   Menghapus kata umum seperti **the**, **is**, **and**, **of**, dan **to**.
5. **Simple stemming**
   Mengubah kata menjadi bentuk dasar sederhana, misalnya:
   documents → document
   searching → search
   systems → system

### 3.15.4 Alur Pembangunan Inverted Index

Token hasil preprocessing digunakan untuk membangun **InvertedIndex**.

Alurnya:
List<String> tokens
↓
InvertedIndex.addDocument(docId, tokens)
↓
Map<String, Set<Integer>>

Sturktur inverted index:
term → posting list

Contoh:
computer → [2, 7, 12, 51]
retrieval → [1, 66, 71, 72]
document → [1, 34, 67, 68]

Penjelasan:
1. Setiap token dimasukkan ke dalam *inverted index*.
2. Jika *term* belum ada, sistem membuat posting list baru.
3. Jika *term* sudah ada, sistem menambahkan **doc_id** ke *posting list*.
4. *Posting list* menggunakan **Set<Integer>** agar dokumen tidak muncul lebih dari sekali untuk *term* yang sama.

### 3.15.5 Alur Pembangunan Trie untuk Wildcard Search

Setelah *inverted index* selesai dibuat, sistem membangun Trie dari vocabulary.

Alurnya:
InvertedIndex.getVocabulary()
↓
Trie.insert(term)

↓
Trie vocabulary

Fungsi Trie:
1. Menyimpan seluruh *vocabulary term* dari *inverted index*.
2. Mendukung pencarian berdasarkan prefix.
3. Digunakan untuk *wildcard query* seperti:
   `comput*`

Contoh:
comput*
↓
prefix = comput
↓
matched terms:
computer
computing
computation
computational

### 3.15.6 Alur Pencarian pada SearchEngine

Class **SearchEngine** menjadi pusat integrasi sistem.

Alur utama pencarian:
User query
↓
SearchEngine.search(query)
↓
Cek jenis query
↓
Jalankan strategi pencarian yang sesuai
↓
Set<Integer> resultIds
↓
printResults(resultIds)

Jenis *query* yang didukung:

| Jenis Query | Contoh | Modul yang Dipakai |
|---|---|---|
| Single-term search | computer | **InvertedIndex** |
| Boolean Retrieval | Information AND retrieval | **BooleanEngine** |

| | | |
|---|---|---|
| Wildcard Search | comput* | **WildcardSearchStrategy** |
| Typo Correction | computr | **EditDistanceStrategy** |

### 3.15.7 Alur Single-Term Search

Single-term search digunakan ketika user memasukkan satu *term* biasa.

Contoh *query*:

computer

Alur:

computer
↓
TextUtil.normalizeToken()
↓
InvertedIndex.containsTerm()
↓
InvertedIndex.getPostingList()
↓
Result doc_id

Penjelasan:

1. *Query* dinormalisasi menggunakan **TextUtil.normalizeToken()**.
2. Sistem mengecek apakah term ada di *inverted index*.
3. Jika ada, sistem mengambil *posting list* dari *term* tersebut.
4. Dokumen hasil pencarian ditampilkan ke terminal.

### 3.15.8 Alur Boolean Retrieval

*Boolean Retrieval* digunakan untuk *query* dengan operator `AND`, `OR`, dan `NOT`.

Contoh *query*:

information AND retrieval

Alur:

Boolean query
↓
BooleanEngine
↓
QueryParser
↓
QueryExpression
↓
evaluate()
↓

Result doc_id

Class yang digunakan:

| Class | Fungsi |
|---|---|
| **QueryExpression** | Interface utama untuk *expression Boolean*. |
| **TermExpression** | Merepresentasikan satu *term*. |
| **AndExpression** | Melakukan operasi intersection. |
| **OrExpression** | Melakukan operasi union. |
| **NotExpression** | Melakukan operasi pengecualian dokumen. |
| **QueryParser** | Mengubah *query* user menjadi *expression tree*. |
| **BooleanEngine** | Menjalankan proses *Boolean Retrieval*. |

Contoh operasi:

information AND retrieval
↓
posting(information) ∩ posting(retrieval)

computer OR software
↓
posting(computer) ∪ posting(software)

computer NOT library
↓
posting(computer) - posting(library)

### 3.15.9 Alur Wildcard Search

*Wildcard* search digunakan untuk *query* yang mengandung tanda “*”.

Contoh *query*:

```
comput*
```

Alur:

comput*
↓
prefix = comput
↓
Trie.getTermsWithPrefix("comput")
↓

matched terms
↓
ambil posting list tiap matched term
↓
gabungkan semua doc_id

Contoh *matched terms*:
computer
computing
computation
computational

Penjelasan:
1. Sistem mengecek apakah *query* mengandung *wildcard*.
2. Sistem hanya memproses *trailing wildcard*, yaitu *wildcard* di akhir kata.
3. Prefix sebelum tanda “*” diambil.
4. Trie mencari semua term yang memiliki prefix tersebut.
5. *Posting list* dari semua *term* yang cocok digabungkan.

### 3.15.10 Alur Edit Distance Correction

*Edit Distance Correction* digunakan ketika *query* tidak ditemukan di *inverted index*.

Contoh *query*:
Computr

Alur:
computr
↓
term tidak ditemukan di InvertedIndex
↓
EditDistanceStrategy
↓
bandingkan dengan vocabulary
↓
Levenshtein Distance terkecil
↓
suggestion = computer
↓
ambil posting list computer

Contoh correction:
computr → computer
retrival → retrieval
documnt → document

Penjelasan:

1. Query dinormalisasi terlebih dahulu.
2. Sistem membandingkan *query* dengan seluruh vocabulary.
3. Sistem menghitung *Levenshtein Distance*.
4. *Term* dengan jarak terkecil dipilih sebagai suggestion.
5. Jika suggestion valid, sistem mengambil *posting list* dari *term* tersebut

### 3.15.11 Alur Output Hasil Pencarian

Setelah hasil pencarian diperoleh, sistem menampilkan dokumen melalui method **printResults()**.

Alur:

```
Set<Integer> resultIds
      ↓
DocumentStore.getDocumentById(id)
      ↓
Document
      ↓
Tampilkan id, title, content
```

Output yang ditampilkan:

```
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems.
```

Jika hasil kosong, sistem menampilkan:

```
No documents found.
```

### 3.15.12 Ringkasan Alur Sistem

Ringkasan keseluruhan alur sistem:

```
data/dokumen.txt
      ↓
DocumentStore
      ↓
BasicPreprocessor
      ↓
InvertedIndex
      ↓
SearchEngine
      ↓
────────────────────────────
 Single-Term Search
 Boolean Retrieval
 Wildcard Search
 Edit Distance Correction
```

↓

Result Document IDs

↓

DocumentStore.getDocumentById()

↓

Output dokumen ke terminal

Dengan arsitektur ini, sistem menjadi lebih modular. Setiap bagian memiliki tanggung jawab masing-masing, sehingga program lebih mudah diuji dan dikembangkan.

# Bab IV
# Pengujian Sistem

## 4.1 Pengujian Load Dataset

```
1|Information Retrieval|Information retrieval is the process of finding relevant documents from a large collection. Search engines use indexing techniques to make retrieval faster and more accurate.
2|Computer Science|Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.
3|Digital Library|A digital library stores books, journals, articles, and research papers in electronic format. Users can search the collection using keywords and retrieve relevant information.
4|Machine Learning|Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.
5|Database System|A database system stores and manages structured data. Users can insert, update, delete, and retrieve information using query languages.
6|Search Engine|A search engine helps users find information on the internet. It uses crawling, indexing, and ranking methods to return relevant web pages.
7|Cybersecurity|Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.
8|Cloud Computing|Cloud computing provides computing resources such as storage, servers, and applications through the internet. Companies use cloud services to reduce infrastructure costs.
9|Software Engineering|Software engineering focuses on designing, developing, testing, and maintaining software systems. It uses structured methods to build reliable and scalable applications.
10|Data Mining|Data mining is the process of discovering useful patterns from large datasets. It is often used in business analytics, customer behavior analysis, and decision support systems.
11|Natural Language Processing|Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.
12|Computer Networks|Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.
13|Operating System|An operating system manages hardware resources and provides services for software applications. It controls memory, processes, files, and input output devices.
14|Web Development|Web development is the process of creating websites and web applications. It involves frontend design, backend programming, databases, and server deployment.
15|Mobile Application|Mobile applications are software programs designed to run on smartphones and tablets. They are used for communication, education, banking, entertainment, and productivity.
16|Artificial Intelligence|Artificial intelligence enables machines to perform tasks that normally require human intelligence. These tasks include reasoning, learning, planning, and problem solving.
17|Data Visualization|Data visualization presents data in graphical forms such as charts, maps, and dashboards. It helps users understand trends, patterns, and comparisons more easily.
18|Information System|An information system combines people, processes, data, and technology to support organizational activities. It helps businesses manage operations and make decisions.
19|Robotics|Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.
20|Human Computer Interaction|Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.
21|Algorithm Design|Algorithm design focuses on creating step by step procedures to solve computational problems. Efficient algorithms can reduce processing time and memory usage.
22|Distributed System|A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.
23|Big Data|Big data refers to extremely large and complex datasets that are difficult to process using traditional tools. It requires special methods for storage, processing, and analysis.
24|Blockchain Technology|Blockchain technology stores data in a distributed and tamper resistant ledger. It is commonly used in cryptocurrency, digital identity, and secure transaction systems.
25|Internet of Things|The internet of things connects physical devices to the internet so they can collect and exchange data. Smart sensors are used in homes, factories, cities, and transportation systems.
26|Data Privacy|Data privacy focuses on protecting personal information from unauthorized access and misuse. Organizations must handle user data responsibly and follow security regulations.
27|Computer Vision|Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.
28|Recommendation System|A recommendation system suggests products, movies, music, or content based on user preferences. It often uses user behavior data and machine learning techniques.
29|Educational Technology|Educational technology uses digital tools to support teaching and learning activities. Online learning platforms help students access materials, quizzes, and virtual classrooms.
30|Bioinformatics|Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.
31|Electronic Commerce|Electronic commerce allows people to buy and sell goods or services through online platforms. It relies on payment systems, product catalogs, user reviews, and secure transactions.
32|Social Media Analytics|Social media analytics examines data from online platforms to understand public opinion and user behavior. It can be used for marketing, trend detection, and sentiment analysis.
33|Knowledge Graph|A knowledge graph represents entities and their relationships in a structured form. It helps search engines and intelligent systems understand connections between concepts.
34|Text Mining|Text mining extracts useful information from unstructured text documents. It can identify keywords, topics, opinions, and relationships in large document collections.
35|Enterprise System|An enterprise system supports business processes across departments such as finance, sales, inventory, and human resources. It helps organizations integrate data and improve efficiency.
36|Data Warehouse|A data warehouse stores historical data from multiple sources for analysis and reporting. It supports business intelligence, dashboards, and decision making.
37|User Experience Design|User experience design focuses on creating products that are useful, usable, and enjoyable. Designers study user needs, workflows, and interface behavior.
38|Application Programming Interface|An application programming interface allows different software systems to communicate with each other. APIs are commonly used to connect web services, mobile applications, and databases.
39|Parallel Computing|Parallel computing divides a large computational task into smaller parts that run at the same time. It can improve performance for simulations, data processing, and scientific computing.
40|Smart City|A smart city uses digital technology and data to improve urban services. Smart traffic systems, public safety platforms, and energy monitoring can make cities more efficient.
41|Network Security|Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.
42|Quantum Computing|Quantum computing uses principles of quantum mechanics to process information. It has potential applications in cryptography, optimization, material science, and complex simulations.
43|Game Development|Game development is the process of designing and building digital games. It includes programming, graphics, physics, sound design, storytelling, and user interaction.
44|Virtual Reality|Virtual reality creates immersive digital environments that users can experience through special devices. It is used in gaming, training, education, and medical simulation.
45|Augmented Reality|Augmented reality adds digital information to the real world through cameras or smart glasses. It can support navigation, education, industrial maintenance, and interactive marketing.
46|DevOps|DevOps combines software development and IT operations to improve delivery speed and system reliability. It uses automation, continuous integration, testing, and monitoring.
```

```
47|Software Testing|Software testing checks whether a program works correctly and meets user requirements. Testing can include unit testing, integration testing, system testing, and acceptance testing.
48|Edge Computing|Edge computing processes data near the source instead of sending everything to a central cloud server. It reduces latency and supports real time applications such as smart cameras and autonomous vehicles.
49|Digital Forensics|Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response.
50|Green Computing|Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.
51|Computer Architecture|Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instructions.
52|Compiler Design|Compiler design studies how programming languages are translated into machine code. A compiler usually performs lexical analysis, syntax analysis, semantic analysis, optimization, and code generation.
53|Information Security|Information security protects data from unauthorized access, modification, disclosure, and destruction. It includes confidentiality, integrity, availability, authentication, and risk management.
54|Cryptography|Cryptography is the science of securing information using mathematical techniques. It is used to protect messages, passwords, digital signatures, and secure online transactions.
55|Data Compression|Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.
56|Multimedia System|A multimedia system manages different types of media such as text, audio, images, animation, and video. It is commonly used in education, entertainment, advertising, and digital communication.
57|Geographic Information System|A geographic information system collects, stores, analyzes, and visualizes spatial data. It is useful for mapping, urban planning, transportation analysis, and environmental monitoring.
58|Decision Support System|A decision support system helps users analyze data and evaluate alternatives. It supports managers in making structured and semi structured decisions.
59|Expert System|An expert system uses knowledge and rules to solve problems in a specific domain. It can support diagnosis, recommendation, planning, and troubleshooting tasks.
60|Pattern Recognition|Pattern recognition identifies regularities or structures in data. It is used in handwriting recognition, speech recognition, medical diagnosis, and image classification.
61|Speech Recognition|Speech recognition converts spoken language into text. It is used in virtual assistants, transcription systems, voice commands, and accessibility tools.
62|Image Processing|Image processing applies computational techniques to improve or analyze images. Common operations include filtering, edge detection, segmentation, enhancement, and object detection.
63|Sentiment Analysis|Sentiment analysis identifies opinions and emotions expressed in text. It is often used to analyze product reviews, social media posts, and customer feedback.
64|Topic Modeling|Topic modeling discovers hidden themes in a collection of documents. It helps summarize large text datasets and organize information automatically.
65|Information Extraction|Information extraction identifies structured information from unstructured text. It can extract names, dates, locations, events, and relationships from documents.
66|Question Answering System|A question answering system returns direct answers to user questions. It combines natural language processing, information retrieval, and knowledge representation.
67|Document Classification|Document classification assigns text documents into predefined categories. It is useful for email filtering, news classification, legal document organization, and academic indexing.
68|Document Clustering|Document clustering groups similar documents without predefined labels. It helps users explore large collections and discover related topics.
69|Web Crawler|A web crawler automatically visits web pages and collects information for indexing. Search engines use crawlers to discover new pages and update existing content.
70|Indexing Technique|Indexing techniques improve the speed of searching in large document collections. An inverted index maps terms to the documents where they appear.
71|Boolean Retrieval|Boolean retrieval uses logical operators to combine search terms. Users can apply AND, OR, and NOT to control which documents are returned.
72|Tolerant Retrieval|Tolerant retrieval allows a search system to handle imperfect queries. It can support spelling correction, wildcard matching, and approximate string matching.
73|Levenshtein Distance|Levenshtein distance measures the minimum number of edits required to change one word into another. It is commonly used for spelling correction and fuzzy matching.
74|Wildcard Search|Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.
75|Posting List|A posting list contains the identifiers of documents that contain a specific term. It is an important component of an inverted index in information retrieval systems.
76|Tokenization|Tokenization splits text into smaller units such as words or terms. It is one of the first steps in text preprocessing and document indexing.
77|Stopword Removal|Stopword removal eliminates common words that may not be useful for retrieval. Words such as the, is, and of are often removed during preprocessing.
78|Stemming|Stemming reduces words to simpler base forms by removing common suffixes. For example, searching, searched, and searches can be reduced to search.
79|Query Processing|Query processing transforms a user query into a form that can be evaluated by a retrieval system. It may include tokenization, parsing, normalization, and operator handling.
80|Ranking Algorithm|A ranking algorithm orders search results based on estimated relevance. Although Boolean retrieval returns matching documents, ranked retrieval can sort documents by importance.
81|Term Frequency|Term frequency counts how often a term appears in a document. It is often used to estimate the importance of a word within a specific document.
82|Inverse Document Frequency|Inverse document frequency measures how rare or common a term is across a document collection. Rare terms are often more useful for distinguishing relevant documents.
83|Vector Space Model|The vector space model represents documents and queries as vectors of term weights. Similarity between a query and a document can be measured using cosine similarity.
84|Cosine Similarity|Cosine similarity measures the angle between two vectors. In text retrieval, it is often used to compare the similarity between a query and a document.
85|Evaluation Metrics|Evaluation metrics are used to measure the quality of a search system. Common metrics include precision, recall, accuracy, F measure, and mean average precision.
86|Precision and Recall|Precision measures how many retrieved documents are relevant, while recall measures how many relevant documents are retrieved. Both metrics are important in information retrieval evaluation.
87|Search Result|A search result is the output returned by a retrieval system after processing a query. It usually contains document identifiers, titles, snippets, and relevance information.
88|Text Preprocessing|Text preprocessing prepares raw text for analysis and indexing. It can include lowercasing, punctuation removal, tokenization, stopword removal, and stemming.
89|Corpus|A corpus is a collection of text documents used for analysis or retrieval. In an information retrieval system, the corpus becomes the searchable document collection.
90|Metadata|Metadata describes information about a document such as title, author, date, category, and source. It can help users filter and understand search results.
91|File Processing|File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.
92|Hash Map|A hash map stores key value pairs and allows fast lookup. In an inverted index, a term can be used as a key and a posting list can be used as the value.
```

```
93|Set Operation|Set operations are useful for Boolean retrieval. Intersection supports AND queries, union supports OR queries, and complement supports NOT queries.
94|Query Parser|A query parser analyzes the structure of a search query. It identifies terms, operators, parentheses, and the correct order of evaluation.
95|Expression Tree|An expression tree represents a query as a tree structure. Each internal node can represent an operator, while each leaf node can represent a search term.
96|Software Modularity|Software modularity divides a program into smaller independent components. A modular search engine is easier to test, maintain, and extend.
97|Design Pattern|A design pattern is a reusable solution to a common software design problem. Patterns such as Facade, Strategy, and Composite can help organize a retrieval system.
98|Command Line Interface|A command line interface allows users to interact with a program using text commands. A simple retrieval system can use a terminal menu to accept search queries.
99|Program Testing|Program testing checks whether each feature works as expected. A search engine should be tested using single term queries, Boolean queries, wildcard queries, and typo queries.
100|System Documentation|System documentation explains how a program works and how users can run it. A good README should describe the dataset format, program features, compile commands, and example queries.
```

```
Index built successfully.
Total documents: 100
Total unique terms: 681
```

## 4.2 Pengujian Inverted Index

```
Sample inverted index:
snippet->[87]
prepare->[88]
smartphone->[15]
accuracy->[85]
tokenization->[76, 79, 88]
without->[68]
these->[16]
robot->[19]
evaluation->[85, 86, 94]
imperfect->[72]
music->[28]
bas->[28, 80]
retrieval->[1, 66, 71, 72, 75, 77, 79, 80, 84, 86, 87, 89, 91, 93, 97, 98]
complex->[23, 42]
rank->[6, 80]
near->[48]
knowledge->[33, 59, 66]
energy->[40, 50]
organizational->[18]
preference->[28]
storytell->[43]
documentation->[100]
raw->[88]
require->[16, 23]
personal->[26]
analysis->[10, 11, 23, 30, 32, 36, 52, 57, 63, 88, 89]
misuse->[26]
secur->[54]
extend->[96]
predefin->[67, 68]
size->[55]
```

Gambar hanya menunjukkan 30 dari 681 inverted index

## 4.3 Pengujian Single-Term Search

Contoh Single-Term Search

Query : computer

Expected : Dokumen-dokumen yang mengandung “computer”

```
Search results for: computer
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.

Doc 4 - Machine Learning
Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.

Doc 7 - Cybersecurity
Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.

Doc 11 - Natural Language Processing
Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.

Doc 12 - Computer Networks
Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.

Doc 19 - Robotics
Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.

Doc 20 - Human Computer Interaction
Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.

Doc 22 - Distributed System
A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.

Doc 27 - Computer Vision
Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.

Doc 30 - Bioinformatics
Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.

Doc 41 - Network Security
Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.

Doc 49 - Digital Forensics
Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response
.

Doc 50 - Green Computing
Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.

Doc 51 - Computer Architecture
Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instr
uctions.

Doc 55 - Data Compression
Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.

Doc 74 - Wildcard Search
Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.
```

## 4.4 Pengujian Boolean Query

Query : information AND retrieval

```
Enter query: information AND retrieval

Search results for: information AND retrieval
Doc 1 - Information Retrieval
Information retrieval is the process of finding relevant documents from a large collection. Search engines use indexing techniques to make retrieval faster and more accurate.

Doc 66 - Question Answering System
A question answering system returns direct answers to user questions. It combines natural language processing, information retrieval, and knowledge representation.

Doc 75 - Posting List
A posting list contains the identifiers of documents that contain a specific term. It is an important component of an inverted index in information retrieval systems.

Doc 86 - Precision and Recall
Precision measures how many retrieved documents are relevant, while recall measures how many relevant documents are retrieved. Both metrics are important in information retrieval evaluation.

Doc 87 - Search Result
A search result is the output returned by a retrieval system after processing a query. It usually contains document identifiers, titles, snippets, and relevance information.

Doc 89 - Corpus
A corpus is a collection of text documents used for analysis or retrieval. In an information retrieval system, the corpus becomes the searchable document collection.
```

Query : computer OR software

```
Enter query: computer OR software

Search results for: computer OR software
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.

Doc 4 - Machine Learning
Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.

Doc 7 - Cybersecurity
Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.

Doc 9 - Software Engineering
Software engineering focuses on designing, developing, testing, and maintaining software systems. It uses structured methods to build reliable and scalable applications.

Doc 11 - Natural Language Processing
Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.

Doc 12 - Computer Networks
Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.

Doc 13 - Operating System
An operating system manages hardware resources and provides services for software applications. It controls memory, processes, files, and input output devices.

Doc 15 - Mobile Application
Mobile applications are software programs designed to run on smartphones and tablets. They are used for communication, education, banking, entertainment, and productivity.

Doc 19 - Robotics
Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.

Doc 20 - Human Computer Interaction
Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.

Doc 22 - Distributed System
A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.

Doc 27 - Computer Vision
Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.

Doc 30 - Bioinformatics
Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.

Doc 38 - Application Programming Interface
An application programming interface allows different software systems to communicate with each other. APIs are commonly used to connect web services, mobile applications, and databases.

Doc 41 - Network Security
Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.

Doc 46 - DevOps
DevOps combines software development and IT operations to improve delivery speed and system reliability. It uses automation, continuous integration, testing, and monitoring.

Doc 47 - Software Testing
Software testing checks whether a program works correctly and meets user requirements. Testing can include unit testing, integration testing, system testing, and acceptance testing.
```

```
Doc 49 - Digital Forensics
Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response
.

Doc 50 - Green Computing
Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.

Doc 51 - Computer Architecture
Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instr
uctions.

Doc 55 - Data Compression
Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.

Doc 74 - Wildcard Search
Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.

Doc 96 - Software Modularity
Software modularity divides a program into smaller independent components. A modular search engine is easier to test, maintain, and extend.

Doc 97 - Design Pattern
A design pattern is a reusable solution to a common software design problem. Patterns such as Facade, Strategy, and Composite can help organize a retrieval system.
```

Query : computer NOT library

```
Search results for: computer NOT library
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.

Doc 4 - Machine Learning
Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.

Doc 7 - Cybersecurity
Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.

Doc 11 - Natural Language Processing
Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.

Doc 12 - Computer Networks
Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.

Doc 19 - Robotics
Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.

Doc 20 - Human Computer Interaction
Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.

Doc 22 - Distributed System
A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.

Doc 27 - Computer Vision
Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.

Doc 30 - Bioinformatics
Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.

Doc 41 - Network Security
Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.

Doc 49 - Digital Forensics
Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response
.

Doc 50 - Green Computing
Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.

Doc 51 - Computer Architecture
Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instr
uctions.

Doc 55 - Data Compression
Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.

Doc 74 - Wildcard Search
Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.
```

## 4.5 Pengujian Wildcard Search

Query : comput*

```
Search results for: comput*
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.

Doc 4 - Machine Learning
Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.

Doc 7 - Cybersecurity
Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.

Doc 8 - Cloud Computing
Cloud computing provides computing resources such as storage, servers, and applications through the internet. Companies use cloud services to reduce infrastructure costs.

Doc 11 - Natural Language Processing
Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.

Doc 12 - Computer Networks
Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.

Doc 19 - Robotics
Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.

Doc 20 - Human Computer Interaction
Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.

Doc 21 - Algorithm Design
Algorithm design focuses on creating step by step procedures to solve computational problems. Efficient algorithms can reduce processing time and memory usage.

Doc 22 - Distributed System
A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.

Doc 27 - Computer Vision
Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.

Doc 30 - Bioinformatics
Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.

Doc 39 - Parallel Computing
Parallel computing divides a large computational task into smaller parts that run at the same time. It can improve performance for simulations, data processing, and scientific computing.

Doc 41 - Network Security
Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.

Doc 42 - Quantum Computing
Quantum computing uses principles of quantum mechanics to process information. It has potential applications in cryptography, optimization, material science, and complex simulations.

Doc 48 - Edge Computing
Edge computing processes data near the source instead of sending everything to a central cloud server. It reduces latency and supports real time applications such as smart cameras and autonom
ous vehicles.

Doc 49 - Digital Forensics
Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response
.
```

```
Doc 50 - Green Computing
Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.

Doc 51 - Computer Architecture
Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instr
uctions.

Doc 55 - Data Compression
Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.

Doc 62 - Image Processing
Image processing applies computational techniques to improve or analyze images. Common operations include filtering, edge detection, segmentation, enhancement, and object detection.

Doc 74 - Wildcard Search
Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.
```

## 4.6 Pengujian Edit Distance

Query : computerr

```
Search results for: computerr
Did you mean: computer
Doc 2 - Computer Science
Computer science is the study of algorithms, data structures, programming languages, and computational systems. It helps people solve problems using logical and systematic methods.

Doc 4 - Machine Learning
Machine learning allows computer systems to learn patterns from data. It is widely used in recommendation systems, image recognition, fraud detection, and natural language processing.

Doc 7 - Cybersecurity
Cybersecurity protects computer systems, networks, and data from digital attacks. It includes encryption, authentication, malware detection, and access control.

Doc 11 - Natural Language Processing
Natural language processing helps computers understand and process human language. It is used in chatbots, machine translation, sentiment analysis, and text classification.

Doc 12 - Computer Networks
Computer networks allow devices to communicate and exchange data. Network protocols define how information is transmitted between computers and servers.

Doc 19 - Robotics
Robotics combines mechanical engineering, electronics, and computer science to build intelligent machines. Robots can perform tasks in manufacturing, healthcare, logistics, and exploration.

Doc 20 - Human Computer Interaction
Human computer interaction studies how people interact with digital systems. Good interface design improves usability, accessibility, and user satisfaction.

Doc 22 - Distributed System
A distributed system consists of multiple computers that work together as a single system. It improves scalability, reliability, and resource sharing.

Doc 27 - Computer Vision
Computer vision allows computers to analyze and understand images or videos. It is used in facial recognition, medical imaging, autonomous vehicles, and quality inspection.

Doc 30 - Bioinformatics
Bioinformatics applies computer science to analyze biological data. It is used in genomics, drug discovery, protein analysis, and medical research.

Doc 41 - Network Security
Network security protects computer networks from unauthorized access, attacks, and data theft. Firewalls, intrusion detection systems, and encryption are common protection methods.

Doc 49 - Digital Forensics
Digital forensics investigates data from computers, mobile devices, and networks to find evidence of cyber incidents. It is used in law enforcement, fraud investigation, and incident response
.

Doc 50 - Green Computing
Green computing focuses on designing and using computer systems in an environmentally responsible way. It aims to reduce energy consumption, electronic waste, and carbon emissions.

Doc 51 - Computer Architecture
Computer architecture describes the design and organization of computer hardware components. It explains how processors, memory, buses, and input output systems work together to execute instr
uctions.

Doc 55 - Data Compression
Data compression reduces the size of files or data streams. It can improve storage efficiency and reduce transmission time across computer networks.

Doc 74 - Wildcard Search
Wildcard search allows users to search using incomplete word patterns. A query such as comput star can match computer, computing, and computation.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.
```

Query “computerr” akan diubah menjadi “computer”

Query : retrival

```
Search results for: retrival
Did you mean: retrieval
Doc 1 - Information Retrieval
Information retrieval is the process of finding relevant documents from a large collection. Search engines use indexing techniques to make retrieval faster and more accurate.

Doc 66 - Question Answering System
A question answering system returns direct answers to user questions. It combines natural language processing, information retrieval, and knowledge representation.

Doc 71 - Boolean Retrieval
Boolean retrieval uses logical operators to combine search terms. Users can apply AND, OR, and NOT to control which documents are returned.

Doc 72 - Tolerant Retrieval
Tolerant retrieval allows a search system to handle imperfect queries. It can support spelling correction, wildcard matching, and approximate string matching.

Doc 75 - Posting List
A posting list contains the identifiers of documents that contain a specific term. It is an important component of an inverted index in information retrieval systems.

Doc 77 - Stopword Removal
Stopword removal eliminates common words that may not be useful for retrieval. Words such as the, is, and of are often removed during preprocessing.

Doc 79 - Query Processing
Query processing transforms a user query into a form that can be evaluated by a retrieval system. It may include tokenization, parsing, normalization, and operator handling.

Doc 80 - Ranking Algorithm
A ranking algorithm orders search results based on estimated relevance. Although Boolean retrieval returns matching documents, ranked retrieval can sort documents by importance.

Doc 84 - Cosine Similarity
Cosine similarity measures the angle between two vectors. In text retrieval, it is often used to compare the similarity between a query and a document.

Doc 86 - Precision and Recall
Precision measures how many retrieved documents are relevant, while recall measures how many relevant documents are retrieved. Both metrics are important in information retrieval evaluation.

Doc 87 - Search Result
A search result is the output returned by a retrieval system after processing a query. It usually contains document identifiers, titles, snippets, and relevance information.

Doc 89 - Corpus
A corpus is a collection of text documents used for analysis or retrieval. In an information retrieval system, the corpus becomes the searchable document collection.

Doc 91 - File Processing
File processing involves reading, writing, and managing files in a computer program. A retrieval system may read document collections from text files before indexing them.

Doc 93 - Set Operation
Set operations are useful for Boolean retrieval. Intersection supports AND queries, union supports OR queries, and complement supports NOT queries.

Doc 97 - Design Pattern
A design pattern is a reusable solution to a common software design problem. Patterns such as Facade, Strategy, and Composite can help organize a retrieval system.

Doc 98 - Command Line Interface
A command line interface allows users to interact with a program using text commands. A simple retrieval system can use a terminal menu to accept search queries.
```

Query : documnt

```
Search results for: documnt
Did you mean: document
Doc 1 - Information Retrieval
Information retrieval is the process of finding relevant documents from a large collection. Search engines use indexing techniques to make retrieval faster and more accurate.

Doc 34 - Text Mining
Text mining extracts useful information from unstructured text documents. It can identify keywords, topics, opinions, and relationships in large document collections.

Doc 64 - Topic Modeling
Topic modeling discovers hidden themes in a collection of documents. It helps summarize large text datasets and organize information automatically.

Doc 65 - Information Extraction
Information extraction identifies structured information from unstructured text. It can extract names, dates, locations, events, and relationships from documents.

Doc 67 - Document Classification
Document classification assigns text documents into predefined categories. It is useful for email filtering, news classification, legal document organization, and academic indexing.

Doc 68 - Document Clustering
Document clustering groups similar documents without predefined labels. It helps users explore large collections and discover related topics.

Doc 70 - Indexing Technique
Indexing techniques improve the speed of searching in large document collections. An inverted index maps terms to the documents where they appear.

Doc 71 - Boolean Retrieval
Boolean retrieval uses logical operators to combine search terms. Users can apply AND, OR, and NOT to control which documents are returned.

Doc 75 - Posting List
A posting list contains the identifiers of documents that contain a specific term. It is an important component of an inverted index in information retrieval systems.

Doc 76 - Tokenization
Tokenization splits text into smaller units such as words or terms. It is one of the first steps in text preprocessing and document indexing.

Doc 80 - Ranking Algorithm
A ranking algorithm orders search results based on estimated relevance. Although Boolean retrieval returns matching documents, ranked retrieval can sort documents by importance.

Doc 81 - Term Frequency
Term frequency counts how often a term appears in a document. It is often used to estimate the importance of a word within a specific document.

Doc 82 - Inverse Document Frequency
Inverse document frequency measures how rare or common a term is across a document collection. Rare terms are often more useful for distinguishing relevant documents.

Doc 83 - Vector Space Model
The vector space model represents documents and queries as vectors of term weights. Similarity between a query and a document can be measured using cosine similarity.

Doc 84 - Cosine Similarity
Cosine similarity measures the angle between two vectors. In text retrieval, it is often used to compare the similarity between a query and a document.

Doc 86 - Precision and Recall
Precision measures how many retrieved documents are relevant, while recall measures how many relevant documents are retrieved. Both metrics are important in information retrieval evaluation.

Doc 87 - Search Result
A search result is the output returned by a retrieval system after processing a query. It usually contains document identifiers, titles, snippets, and relevance information.
```

## 4.7 Screenshot Program

```
Index built successfully.
Total documents: 100
Total unique terms: 681

===== SIMPLE IR SEARCH ENGINE =====
1. Search query
2. Show sample inverted index
3. Exit
Choose menu: 
```

# Bab V
# Cara Menjalankan Program

## 5.1 Persiapan Program

Sebelum program dijalankan, pengguna perlu memastikan bahwa JDK sudah terpasang pada komputer. JDK diperlukan karena program ini dibuat menggunakan bahasa Java dan perlu dikompilasi terlebih dahulu sebelum dijalankan.

Untuk mengecek apakah java sudah terpasang, pengguna dapat membuka terminal atau command prompt, lalu menjalankan perintah berikut:

```
java -version
javac -version
```

Jika kedua perintah tersebut menampilkan versi Java, maka komputer sudah siap digunakan untuk menjalankan program.

Struktur folder program harus berada dalam bentuk berikut:

```
IR_Search_Engine/
├── data/
│   └── dokumen.txt
├── src/
│   ├── Main.java
│   ├── core/
│   ├── preprocessing/
│   ├── booleanquery/
│   ├── tolerant/
│   └── util/
└── README.md
```

Program harus dijalankan dari folder utama project, yaitu folder yang memiliki folder **src** dan **data**. Folder **data** tidak perlu dimasukkan ke dalam folder **src**, karena file **dokumen.txt** akan dibaca melalui path:

```
data/dokumen.txt
```

## 5.2 Compile Program

Sebelum dijalankan, seluruh file Java perlu dikompilasi. Proses compile bertujuan untuk mengubah file **.java** menjadi file **.class** yang dapat dijalankan oleh Java.

### 5.2.1 Compile di Windows CMD

Pada Windows, buka Command Prompt, lalu masuk ke folder utama project.

Contoh:

```
cd ProjectFolder
```

atau:
cd C:\Users\NamaUser\Documents\IR_Search_Engine

Setelah berada di folder utama project, buat folder **out** sebagai tempat hasil compile:
mkdir out

Kemudian compile program dengan perintah:
javac -sourcepath src -d out src\Main.java

Perintah tersebut akan meng-compile **Main.java** beserta class lain yang digunakan di dalam package **core**, **preprocessing**, **booleanquery**, **tolerant**, dan **util**.

Jika proses compile berhasil, maka folder **out** akan berisi file **.class** hasil kompilasi.

### 5.2.2 Compile di Mac atau Linux Terminal

Pada Mac atau Linux, buka terminal, lalu masuk ke folder utama project.

Contoh:
cd ProjectFolder

atau:
cd ~/Documents/IR_Search_Engine

Buat folder **out** untuk menyimpan hasil compile:
mkdir -p out

Kemudian compile seluruh file Java dengan perintah:
javac -d out $(find src -name "*.java")

Perintah tersebut akan mencari seluruh file **.java** di dalam folder **src**, lalu meng-compile semuanya ke dalam folder out.

## 5.3 Menjalankan Program

Setelah proses compile selesai, program dapat dijalankan menggunakan perintah berikut:
java -cp out Main

Perintah tersebut menjalankan class **Main** dari hasil compile yang berada di folder **out**.

Jika program berhasil dijalankan, maka akan muncul informasi bahwa index berhasil dibangun, jumlah dokumen, jumlah vocabulary, dan menu utama program.

Contoh tampilan awal program:
Index built successfully.
Total documents: 100

```
Total unique terms: 681

===== SIMPLE IR SEARCH ENGINE =====
1. Search query
2. Show sample inverted index
3. Exit
Choose menu:
```

Menu program terdiri dari tiga pilihan utama:
1. Search query
2. Show sample inverted index
3. Exit

Pilihan **Search query** digunakan untuk memasukkan query pencarian. Pilihan **Show** sample **inverted index** digunakan untuk menampilkan sebagian isi *inverted index*. Pilihan **Exit** digunakan untuk keluar dari program.

# BAB VI
# Kesimpulan

## 6.1 Kesimpulan

Berdasarkan implementasi yang telah dilakukan, sistem Information Retrieval sederhana berhasil dibuat menggunakan bahasa Java. Sistem mampu membaca 100 dokumen dari file teks, melakukan preprocessing, membangun inverted index, dan melakukan pencarian dokumen berdasarkan query pengguna.

Selain pencarian single-term, sistem juga telah mendukung tolerant retrieval. Wildcard search berhasil diimplementasikan menggunakan Trie untuk query trailing wildcard seperti `comput*`. Selain itu, spelling correction berhasil diimplementasikan menggunakan Levenshtein Distance untuk menangani typo seperti `computr` menjadi `computer`.

Class **SearchEngine** digunakan sebagai facade agar proses pencarian dapat dijalankan melalui satu pintu utama. Dengan demikian, class **Main** tidak perlu memanggil seluruh komponen internal secara langsung. Class **SearchEngine** bertugas mengatur alur pencarian, mulai dari *single-term search*, *Boolean Retrieval*, *wildcard search*, hingga *edit distance correction*.

Secara keseluruhan, sistem yang dibuat telah memenuhi tujuan utama tugas, yaitu membangun *search engine* sederhana berbasis Java dengan *preprocessing*, *inverted index*, *boolean retrieval*, dan *tolerant retrieval*.

## 6.2 Keterbatasan Sistem

Meskipun sistem sudah berhasil menjalankan fitur utama *Information Retrieval*, masih terdapat beberapa keterbatasan. Pertama, *wildcard search* hanya mendukung *trailing wildcard* seperti `comput*`, sehingga belum dapat menangani *leading wildcard* seperti `*tion` atau *middle wildcard* seperti `comp*ion`. Kedua, stemming yang digunakan masih sederhana dan berbasis aturan suffix, sehingga hasilnya belum selalu sempurna untuk semua bentuk kata. Ketiga, sistem belum menerapkan ranking dokumen, sehingga hasil pencarian hanya berupa daftar dokumen yang cocok, bukan dokumen yang diurutkan berdasarkan tingkat relevansi.. Selain itu, program masih menggunakan antarmuka terminal dan belum memiliki tampilan grafis atau web.

## 6.3 Saran Pengembangan

Pengembangan selanjutnya dapat dilakukan dengan menambahkan *ranking* dokumen menggunakan *TF-IDF* atau *Vector Space Model* agar hasil pencarian dapat diurutkan berdasarkan relevansi. Selain itu, *wildcard search* dapat diperluas agar mendukung *leading wildcard* dan *middle wildcard*. Fitur *spelling correction* juga dapat dibuat lebih efisien menggunakan *k-gram index*, sehingga sistem tidak perlu membandingkan *query* dengan seluruh vocabulary. Program juga dapat dikembangkan dengan antarmuka grafis atau web agar lebih mudah digunakan.